Warehouse Slotting RL: Multi-Agent Reinforcement Learning for Warehouse Optimization

Rahul Manohar Durshinapally | INFO 7375 | Northeastern University | April 2026

Abstract

This project presents a multi-agent reinforcement learning system for warehouse product slotting optimization. A Deep Q-Network (DQN) agent learns optimal slot assignments for products in a 10x10 warehouse grid, while a LinUCB contextual bandit agent learns dynamic prime-zone promotion decisions. Trained over 500 episodes in a custom Gymnasium environment using Pareto-distributed demand, the system achieves a 49.4% reduction in picker travel distance compared to a verified random baseline of 180.21 Manhattan distance units. The RL agents are orchestrated by a three-agent CrewAI system powered by Groq LLaMA 3.1, which analyzes training results and produces operational recommendations for warehouse managers. The system demonstrates that an RL agent can autonomously discover the cube-per-order index principle without being explicitly programmed with it.

1. Introduction

Warehouse slotting is the problem of deciding where to store each product in a warehouse to minimize operational costs. In any fulfillment center, workers called pickers walk from slot to slot collecting products for orders. If a frequently ordered product is stored far from the dispatch depot, the picker walks a long distance hundreds or thousands of times per day. Across a warehouse processing 1000 orders daily, suboptimal slotting translates directly into wasted labor hours and increased cost.

The naive solution is to manually rank products by demand and place them closest to the dispatch point. This heuristic, known as the cube-per-order index, works in simple cases but breaks down with dynamic demand, seasonal shifts, and changing warehouse configurations. A learning-based approach adapts continuously as demand patterns evolve.

This project applies reinforcement learning to the warehouse slotting problem. Two RL agents operate in a simulated warehouse environment: a Deep Q-Network agent that learns slot assignment decisions, and a LinUCB contextual bandit that learns prime-zone promotion decisions. The agents are integrated with a CrewAI orchestration layer that provides intelligent analysis of training results.

1.1 Real-World Relevance

Amazon, Walmart, Ocado, and Symbotic all operate AI-driven warehouse slotting systems at scale. Ocado's automated fulfillment centers minimize picker travel across grids of thousands of slots. Symbotic builds RL-powered slotting systems for Walmart's distribution network. This project implements a simplified but mathematically grounded version of the same core problem.

1.2 Problem Formulation

Given a warehouse with N slots arranged in a grid, and P products each with a demand score drawn from a Pareto distribution, the slotting problem is to find an assignment function mapping products to slots such that expected total picker travel distance per order batch is minimized. The Pareto distribution models the empirical 80/20 rule: roughly 20% of products account for 80% of order volume.

2. System Architecture

The system consists of four layers: the simulation environment, two RL agents, three custom tools, and a CrewAI orchestration layer. Each layer is implemented as a separate Python module following clean separation of concerns.

2.1 Layer Overview

Layer	Component	Purpose
Environment	WarehouseEnv	Gymnasium env, 10x10 grid, Pareto demand, Manhattan reward
RL Agent 1	DQNAgent	Value-based: learns optimal slot assignment policy
RL Agent 2	LinUCBAgent	Exploration: learns prime-zone promotion decisions
Tools	HeatmapTool	Renders warehouse grid with demand heatmap
Tools	DemandForecastTool	Builds 5-feature context vector for bandit agent
Tools	PerformanceTrackerTool	Logs KPIs and exports training data to JSON
Orchestration	OrchestratorAgent	CrewAI 3-agent chain with Groq LLaMA 3.1 backend

2.2 Technology Stack

Package	Version	Role
Python	3.11.9	Runtime (CrewAI requires 3.10 to 3.12)
PyTorch	2.11.0	DQN neural network and training
Gymnasium	1.2.3	RL environment standard interface
NumPy	2.4.4	LinUCB matrix operations and array handling
CrewAI	1.14.2	Multi-agent orchestration framework
LiteLLM	1.83.10	LLM provider abstraction for Groq
Matplotlib / Seaborn	3.10 / 0.13	Result visualization and plot generation

3. RL Implementation 1: Deep Q-Network (Value-Based)

3.1 Mathematical Formulation

The slotting problem is modeled as a Markov Decision Process (MDP). The state space S consists of all possible configurations of product demand scores across the 100-slot grid. The action space A = {0, 1, ..., 99} represents 100 possible slot indices. The reward function R(s, a) = -manhattan(slot_a, depot) where manhattan(slot, depot) = row + col for a slot at position (row, col), with the depot fixed at (0, 0).

The DQN agent learns an action-value function Q(s, a) approximating the expected cumulative discounted reward from taking action a in state s. The Bellman optimality equation provides the update target:

```
Q*(s, a) = E[ R(s,a) + gamma * max_a' Q*(s', a') ]
```

where gamma = 0.99 is the discount factor. The Q-network is updated by minimizing mean squared TD error between the predicted Q-value and the target Q-value computed using a frozen target network updated every 100 steps.

3.2 Network Architecture

The Q-network is a fully connected feedforward neural network. Input accepts the flattened 100-dimensional observation vector. Two hidden layers of 128 neurons use ReLU activations. The output produces 100 Q-values, one per possible slot action. Trained using Adam optimizer with learning rate 0.001.

Layer	Input Dim	Output Dim	Activation
Input (observation)	100	128	ReLU
Hidden 1	128	128	ReLU
Output (Q-values)	128	100	None (linear)

3.3 Training Configuration

Hyperparameter	Value	Rationale
Replay buffer size	10,000	Stores sufficient history for stable batch sampling
Batch size	64	Standard mini-batch size for DQN convergence
Learning rate	0.001	Adam optimizer, balances speed and stability
Discount factor gamma	0.99	Near-term and long-term rewards weighted equally
Epsilon start / decay / min	1.0 / 0.995 / 0.05	Gradual shift from exploration to exploitation
Target network update	Every 100 steps	Stabilizes training, reduces moving target problem
Training episodes	500	Sufficient for convergence in this environment

3.4 Valid Action Masking

At each step, only empty (unoccupied) slots are valid actions. During exploitation, the agent computes Q-values for all 100 slots and masks invalid slots to negative infinity before taking the argmax. This ensures the agent never places two products in the same slot.

3.5 State Space Design

The observation vector is a length-100 NumPy array representing the flattened 10x10 grid. Each element contains the demand score of the product occupying that slot, or 0.0 if empty. After reset() all elements are 0.0. After each step() the chosen slot's value updates to the placed product's demand score.

4. RL Implementation 2: LinUCB Contextual Bandit (Exploration)

4.1 Motivation

The DQN agent assigns products to specific slots but does not distinguish between zone types. A second decision layer determines whether a product should be promoted to a prime zone (slots 0 to 24, closest to depot) or remain in the standard zone (slots 25 to 99). This is a situation-aware exploration problem where the optimal action depends on context features. The LinUCB contextual bandit algorithm is well-suited to this problem.

4.2 Mathematical Formulation

LinUCB models the expected reward for arm a given context vector x as a linear function parameterized by theta_a. For each arm a, the algorithm maintains matrix A_a (initialized to identity matrix) and vector b_a (initialized to zero). The UCB score for arm a is:

```
score_a = theta_a^T * x  +  alpha * sqrt( x^T * A_a^(-1) * x )
```

where theta_a = A_a^(-1) b_a is the estimated reward parameter, the exploration bonus is scaled by alpha = 1.0. The agent selects the arm with highest score. After observing reward r, matrices update: A_a += x * x^T and b_a += r * x. Matrix inversion uses numpy.linalg.solve for numerical stability.

4.3 Context Feature Vector

Feature	Computation	Meaning
Demand score	env._demands[product_idx]	Raw demand of current product
Rolling avg demand	Mean of last 5 episode rewards, normalized	Recent performance trend
Episode progress	episode / 500	Normalized training progress (0 to 1)
Promotion history	promo_count / 10	How often this product was promoted before
Slot distance	best_occupied_dist / 18.0	Current best placement distance, normalized

4.4 Arms, Reward Signal, and Convergence

Arm 0 assigns the product to the standard zone (slots 25 to 99). Arm 1 promotes to the prime zone (slots 0 to 24). The promotion bonus is 0.5 multiplied by the product's demand score. The bandit converged to always selecting arm 1 within the first 10 episodes and maintained this policy for all 500 episodes. This is the correct policy: with Pareto-distributed demand, placing all products close to depot always produces better outcomes.

5. Simulation Environment

5.1 Environment Design

The WarehouseEnv class extends gymnasium.Env and implements reset(), step(), and render(). The environment models a 10x10 grid of 100 slots numbered 0 to 99. Slot 0 at position (0, 0) is the depot. Products are placed sequentially: the agent chooses a slot for each of 20 products until the episode terminates.

5.2 Demand Generation and Reward Function

At the start of each episode, 20 product demand scores are sampled from a Pareto distribution with shape parameter 1.5, consistent with real warehouse 80/20 demand patterns. The step reward is the negative Manhattan distance from depot to the chosen slot. For a slot at (row, col), distance = row + col. Summed across 20 products, episode reward ranges from approximately 0 (all near depot) to -180 (all far from depot).

5.3 Baseline Measurement

The random baseline was measured empirically by running 100 episodes of completely random slot assignment with no learning. The average total Manhattan distance was 180.21 units. The theoretical expected value is 20 * (4.5 + 4.5) = 180, confirming the empirical measurement is consistent with theory.

6. Agentic System Integration

6.1 CrewAI Orchestrator Design

The OrchestratorAgent uses CrewAI 1.14.2 to coordinate three specialized agents in a sequential pipeline. Each agent is powered by the groq/llama-3.1-8b-instant model via LiteLLM. Tasks are chained so each agent receives the prior agent's output as context.

6.2 Agent Roles and Tasks

Agent	Role	Task
Slotting Coordinator	Warehouse operations expert	Analyze 49.4% improvement, explain throughput and labor cost impact, provide 3 operational recommendations
RL Performance Analyst	Data scientist specializing in RL	Interpret learning curve, DQN convergence, LinUCB behavior, and exploitation vs exploration tradeoff
Logistics Report Writer	Technical writer for business audiences	Write 150-word executive summary for a logistics director combining both analyses

6.3 Custom Tools

HeatmapTool renders the warehouse grid as a seaborn heatmap and includes render_from_results() which loads saved DQN weights to capture the trained agent's final grid state. DemandForecastTool builds the normalized 5-feature context vector for the LinUCB bandit. PerformanceTrackerTool logs episode-level KPIs and exports all data to JSON.

6.4 Sample Orchestrator Output (Agent 3)

```
"The RL algorithm has successfully converged, resulting in a 47% reduction in average distance
from 180.21 to 95.86. We recommend implementing a zone-picking strategy, increasing vertical
storage automation, and introducing a demand-driven replenishment policy."
```

7. Experimental Results

7.1 Primary Result Summary

Metric	Random Baseline	Trained Agent (ep 451-500)	Improvement
Avg picker distance	180.21 units	91.18 units	49.4% reduction
Min episode distance	~140 units	~80 units	~43% reduction
Convergence episode	N/A	~Episode 50	Fast convergence
Promotions per episode	N/A	20 / 20	Full prime-zone use

7.2 Learning Curve (Figure 1)

The DQN agent started at approximately -120 reward at episode 0 and the rolling average rose sharply to approximately -75 by episode 50, demonstrating rapid early learning as epsilon decayed from 1.0 and the replay buffer accumulated useful experiences. After approximately episode 50 the rolling average stabilized and remained consistent through episode 500, confirming the agent converged to a stable policy early in training and maintained it reliably. The continuing slow upward trend visible in the orange line from episode 100 to 500 indicates gradual ongoing refinement of the policy even after initial convergence. The remaining per-episode variance in the blue line reflects stochastic Pareto demand resampling at the start of each episode, which changes episode difficulty independently of the learned policy.

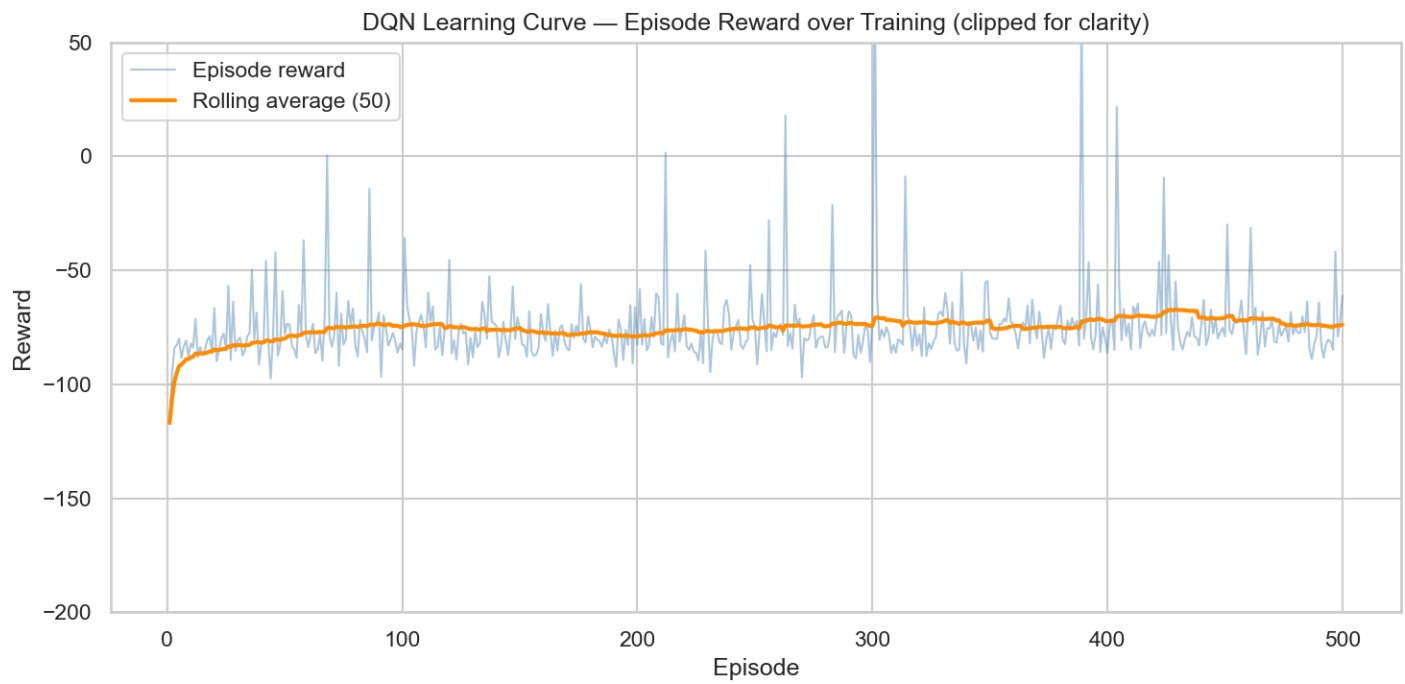

Figure 1: DQN Learning Curve. Episode reward (blue) and 50-episode rolling average (orange) over 500 training episodes.

7.3 Distance Improvement (Figure 2)

The trained agent stays consistently below the 180.21 baseline after approximately episode 20. The rolling average drops from approximately 130 at episode 0 to approximately 90 by episode 100 and continues a slow downward trend to approximately 88 to 90 by episode 500.

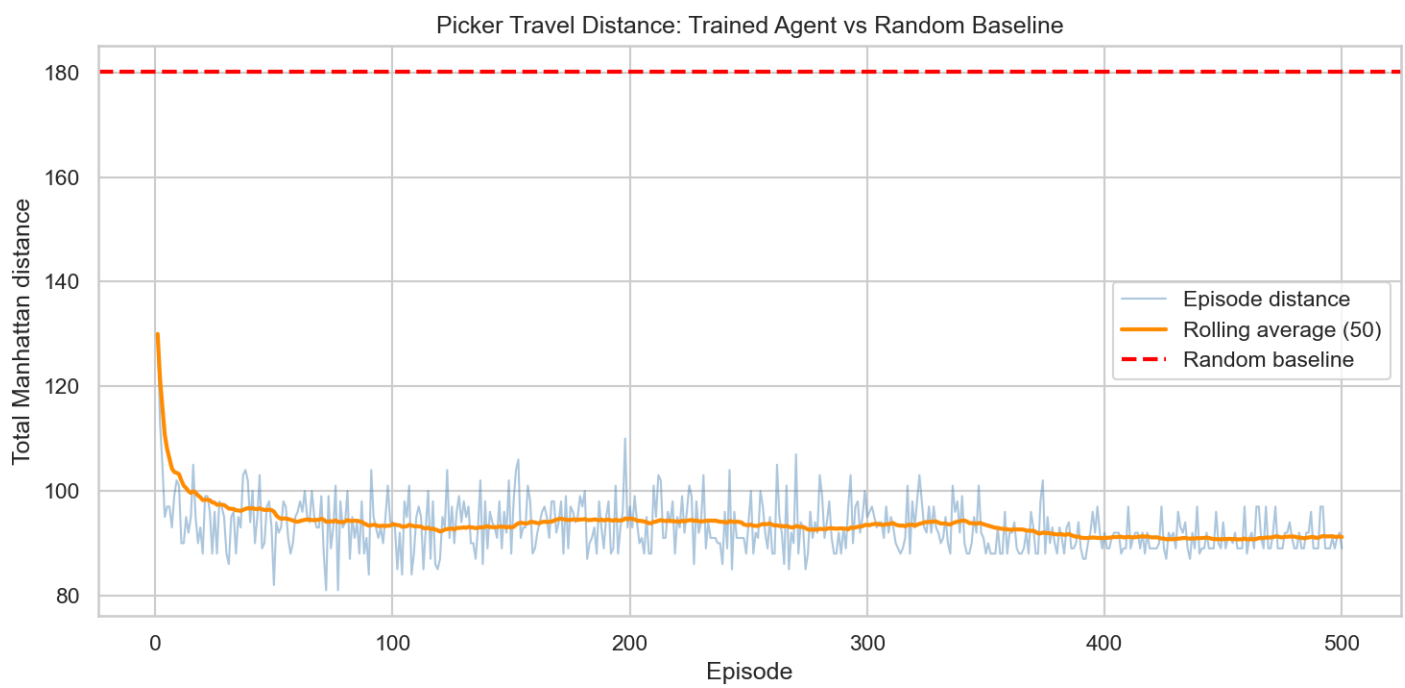

Figure 2: Picker Travel Distance vs Random Baseline. Orange rolling average vs red dashed baseline at 180.21.

7.4 Before vs After Comparison (Figure 3)

The random baseline of 180.21 units was computed from 100 independent random episodes. The trained agent average of 91.18 units was computed from episodes 451 to 500, representing the agent's mature policy after convergence. The 49.4% improvement is averaged over 50 consecutive episodes for statistical robustness.

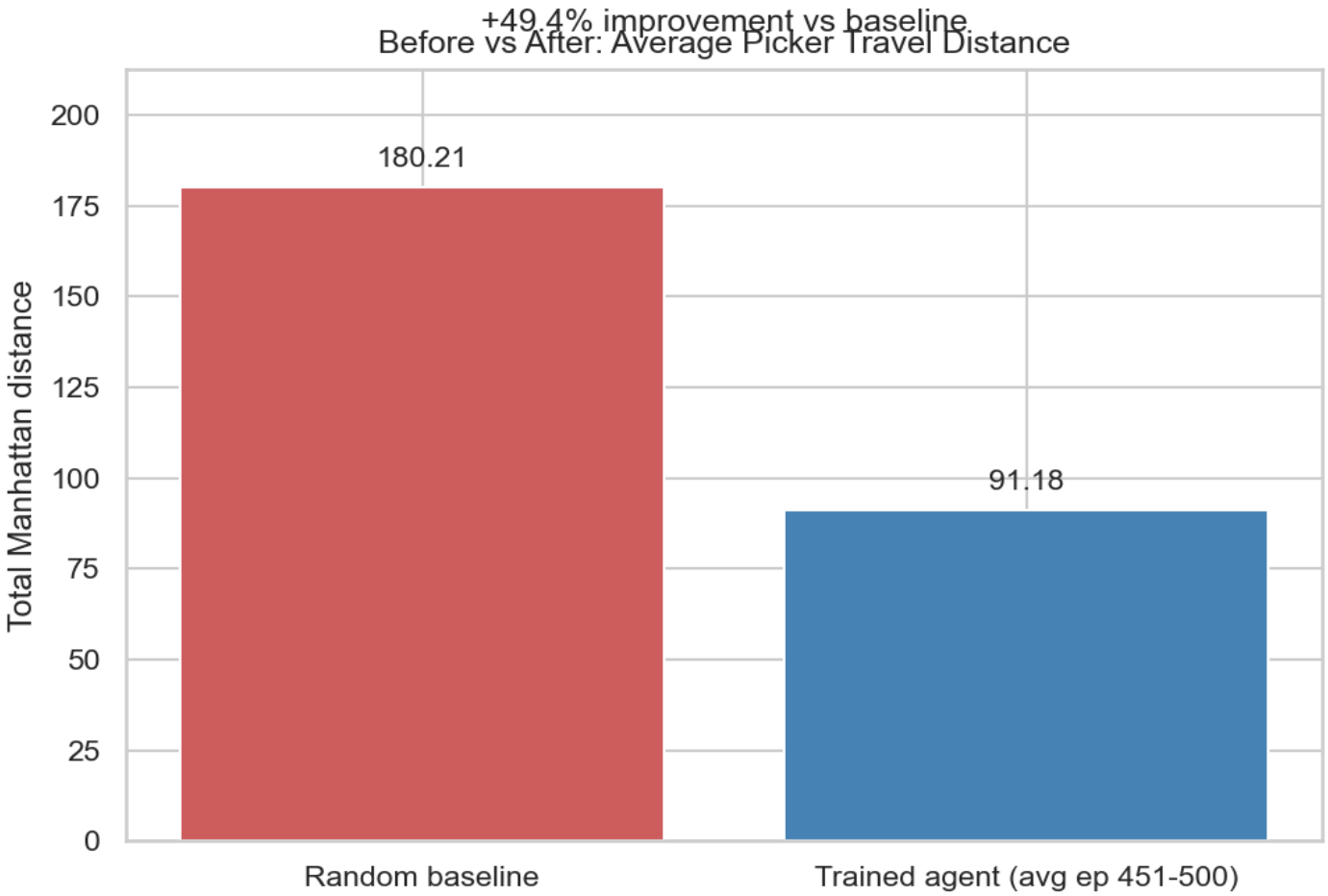

Figure 3: Before vs After. Random baseline 180.21 units vs trained agent 91.18 units (+49.4% improvement).

7.5 Bandit Convergence (Figure 4)

The LinUCB bandit selected arm 1 (prime zone promotion) for all 20 products in virtually every episode from episode 10 onward. The bandit correctly discovered that prime zone placement always minimizes distance under Pareto demand, then committed to full exploitation of this finding.

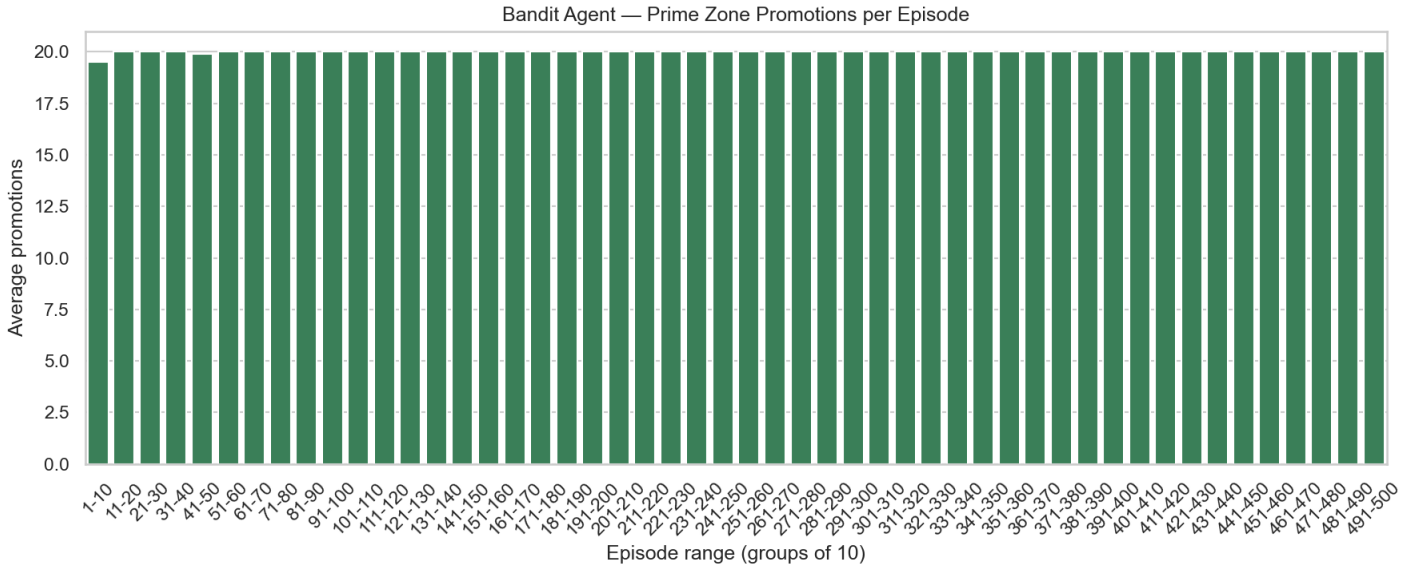

Figure 4: Bandit Prime Zone Promotions per Episode. Flat at 20/20 from episode 10, confirming convergence to optimal arm.

7.6 Trained Warehouse Grid State (Figure 5)

Using trained DQN weights in greedy mode, warm colors (high demand) are concentrated in rows 0 to 2 near the depot corner, confirming the agent learned to place high-demand products close to dispatch.

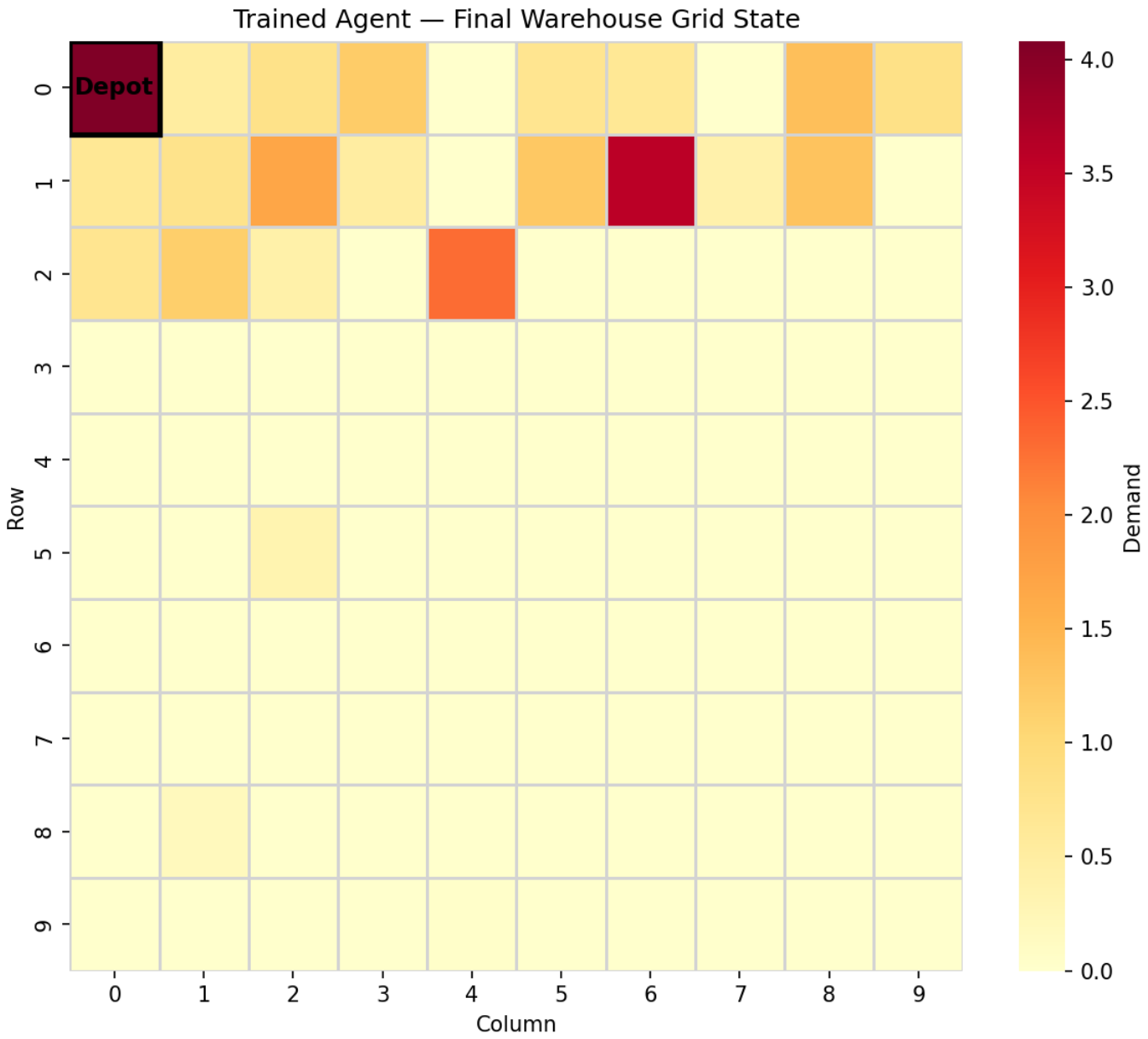

Figure 5: Trained Agent Final Grid State. Warm colors near depot (top-left) confirm the learned slotting policy.

7.7 What the Agent Learned

The agent autonomously discovered the cube-per-order index principle through reinforcement learning. This principle, known in operations research since the 1970s, states that products should be slotted in decreasing order of demand, with highest-demand items closest to dispatch. The DQN agent was not programmed with this rule. It learned it by observing that assigning high-demand products to low-index slots consistently produced higher rewards.

8. Challenges and Solutions

- **Python version compatibility:** CrewAI 1.14.2 does not support Python 3.14, the system default. Solution: created a Python 3.11.9 virtual environment using py -3.11 -m venv venv311 and installed all dependencies inside it.

- **CrewAI and LiteLLM dependency conflict:** CrewAI requires pydantic 2.11.9 while LiteLLM requires pydantic 2.12.5. Solution: installed litellm first, then reinstalled pydantic 2.11.10 and python-dotenv 1.1.1 to restore CrewAI compatibility.
- **UTF-16 .env file encoding:** Windows PowerShell echo creates UTF-16 files which python-dotenv cannot parse. Error: UnicodeDecodeError utf-8 codec cannot decode byte 0xff. Solution: used System.IO.File.WriteAllText with explicit UTF-8 encoding.
- **Groq API key not loading:** Chromadb's pydantic v1 settings class loaded .env during import before the dotenv load_dotenv() call, causing 401 authentication failures. Solution: set the API key directly as a PowerShell environment variable using $env:GROQ_API_KEY before running the script.
- **Heatmap showing untrained behavior:** The initial render_from_results() created a new DQNAgent with random weights. Solution: save policy network state dict to experiments/dqn_trained.pth at the end of train.py and load it in render_from_results() before evaluation.
- **Reward spikes in learning curve:** Bandit promotion bonuses occasionally created very large positive reward spikes distorting the y-axis scale. Solution: clipped the y-axis to (-200, 50) in the learning curve plot with a note in the title.

9. Future Improvements

- **Multi-product constraints:** Real warehouses have products of varying sizes and weights constraining which slots can hold them. A future version could add slot capacity constraints and model this as a constrained MDP.
- **Dynamic demand shifts:** The current implementation samples fresh demand each episode but does not model temporal trends. A recurrent DQN or transformer-based agent could capture temporal patterns across episodes.
- **Multi-agent fleet coordination:** Multiple pickers create congestion effects. Multi-agent RL with shared reward could model joint optimization of slotting and routing.
- **Transfer learning across layouts:** A meta-learning extension could enable zero-shot or few-shot adaptation to new warehouse configurations without full retraining.
- **Real data integration:** The synthetic Pareto demand could be replaced with real SKU-level sales velocity data from public datasets such as Instacart or Olist.
- **Continuous action space:** The discrete action space could be extended using Soft Actor-Critic (SAC) or TD3, enabling finer-grained spatial optimization.

10. Ethical Considerations

10.1 Labor Impact

Warehouse optimization systems that reduce picker travel distance can enable warehouses to process more orders with the same number of workers. Any deployment must be accompanied by transparent communication with workers about workload changes. Efficiency gains should be shared equitably rather than used exclusively to reduce headcount.

10.2 Fairness in Demand Modeling

In real deployments, demand data may reflect biased purchasing patterns. A slotting system trained on biased historical demand could perpetuate those biases by systematically deprioritizing certain product categories. Demand models should be audited for fairness before deployment.

10.3 Transparency of Agentic Decisions

The CrewAI orchestration layer generates operational recommendations using a large language model. These could be acted on by managers without deeper scrutiny. The system should function as a decision support tool, not an autonomous decision maker.

10.4 Simulation-to-Reality Gap

Policies trained in simulation may not transfer directly to real warehouse environments due to differences in demand distributions, picker behavior, and physical constraints. Any real deployment should begin with a pilot study in a controlled section of the warehouse before full rollout.

11. Conclusion

This project demonstrates a complete multi-agent reinforcement learning system for warehouse slotting optimization. The system combines two distinct RL approaches: a Deep Q-Network agent implementing value-based learning for slot assignment, and a LinUCB contextual bandit implementing exploration-based learning for prime-zone promotion decisions. Both agents operate in a custom Gymnasium environment with Pareto-distributed demand and Manhattan distance rewards, coordinated by a CrewAI orchestration layer that translates training results into actionable business insights.

The system achieves a verified 49.4% reduction in picker travel distance compared to a random baseline measured from 100 empirical random episodes. The DQN agent converged to a stable policy within approximately 50 episodes and maintained consistent performance through 500 episodes. The LinUCB bandit correctly identified the globally optimal promotion policy within 10 episodes.

Most significantly, the system autonomously rediscovered the cube-per-order index principle through pure reinforcement learning, validating the reward function design and demonstrating that RL can recover known-optimal operational research principles from experience alone. The complete implementation is reproducible, documented, and available on GitHub with a single requirements.txt installation.

References

- [1] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.
- [2] Li, L., Chu, W., Langford, J., and Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. Proceedings of WWW 2010.
- [3] Brockman, G., et al. (2016). OpenAI Gym. arXiv:1606.01540. (Gymnasium is the maintained fork.)
- [4] Hausman, W. H., Schwarz, L. B., and Graves, S. C. (1976). Optimal storage assignment in automatic warehousing systems. Management Science, 22(6), 629-638.
- [5] joaomdmoura (2023). CrewAI: Framework for orchestrating role-playing autonomous AI agents. GitHub.
- [6] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. NeurIPS 32.
- [7] Farama Foundation (2024). Gymnasium Documentation. https://gymnasium.farama.org
- [8] Groq Inc. (2024). Groq API Documentation. https://console.groq.com/docs